

Claude-Optimized Build Specification

Spreadsheet-First Metadata Catalog (MVP)

1. Product Goal

Build a lightweight metadata catalog that lets invited users:

- Open a workspace via invite link
- Explore a demo dataset
- Upload metadata via CSV
- Document tables and columns in a spreadsheet-style editor
- Define business terms
- Track documentation coverage

The system must scale to 100k+ metadata rows.

MVP constraints:

- No authentication providers
- No external database connectors
- Access via invite links only

2. Technology Stack

Frontend

- React
- TypeScript
- Vite
- Tailwind CSS
- TanStack Table

Backend

- ASP.NET Core
- Minimal APIs

Database

- PostgreSQL

3. Architecture Principles

- Use explicit relational tables
- Avoid generic asset graphs
- Use UUID primary keys
- Add foreign keys and indexes
- Avoid JSON metadata blobs
- Separate read models and write models
- Optimize for fast metadata grid queries

4. Backend Architecture

Request Flow

Endpoints → Services → Repositories → Database

Endpoints

Handle HTTP requests and validation.

Services

Contain business logic and orchestration.

Repositories

Handle database queries only.

5. Session Authentication

All API requests require a valid session cookie.

Cookie name:

participant_session

Session duration:

30 days

Sessions use rolling expiration.

Each authenticated request must update:

participant_sessions.last_seen_at

If last_seen_at is older than 30 days:

→ session is expired

→ return HTTP 401 Unauthorized

Middleware responsibilities:

1. Read cookie
2. Look up participant_sessions table
3. Verify session not expired

4. Update last_seen_at

5. Attach participant_id and workspace_id to request context

6. Invite Access Flow

Invite URL:

/invite/{token}

Invite Flow

User opens invite

→ Invite validated

→ Email collected

→ Participant upserted

→ Session cookie issued

→ Workspace opened

7. Participant Upsert and Invite Usage Rules

Endpoint:

POST /invite/{token}/join

Participants are unique by:

(workspace_id, email)

Behavior:

If a participant with the same email already exists
in the workspace:

- reuse the existing participant
- update last_seen_at
- DO NOT increment invites.used_count

If no participant exists:

- create a new participant
- increment invites.used_count

This ensures returning users keep their history while
invite usage limits still function correctly.

8. Invite Validation Rules

Invite invalid if:

`expires_at < now()`

OR

`used_count >= max_uses`

Errors

404 → invite not found

410 → invite expired

429 → invite usage exceeded

9. Database Schema

workspaces

id

name

created_at

invites

id

workspace_id

token

created_at

expires_at

max_uses

used_count

participants

id

workspace_id

email

invite_id

created_at

last_seen_at

Unique constraint:

(workspace_id, email)

participant_sessions

id

participant_id

workspace_id

created_at

last_seen_at

schemas

id

workspace_id

name

tables

id

workspace_id

schema_id

name

description

created_at

updated_at

columns

id

workspace_id

table_id

name

data_type
example_value
description
owner
version
created_at
updated_at

Column version is used for optimistic concurrency control.

PATCH requests must include the current version.

If version mismatch occurs → return 409 Conflict.

relationships
id
workspace_id
source_column_id
target_column_id
relationship_type

business_terms
id
workspace_id
name
definition

term_column_mapping

id

term_id

column_id

metadata_changes

id

entity_type

entity_id

field

old_value

new_value

participant_id

edited_at

10. Projection Table (Read Model)

Table name:

column_catalog_editor

Purpose:

Provide fast spreadsheet grid queries.

This table is NOT a materialized view.

It is a writable projection table maintained by the application.

Fields

workspace_id

column_id
schema_name
table_name
column_name
data_type
example_value
description
business_term
owner

11. Metadata Grid Query

Endpoint:

GET /metadata/columns

IMPORTANT:

The grid must read from column_catalog_editor.

Example query:

```
SELECT *  
FROM column_catalog_editor  
WHERE workspace_id = ?  
LIMIT 200
```

12. Pagination Contract

GET /metadata/columns

Query parameters:

limit (default 200)

offset

search

undocumented_only

Example:

GET /metadata/columns?limit=200&offset=0

13. CSV Metadata Upload

Endpoint:

POST /metadata/upload

Upload flow

Upload CSV

→ Parse metadata

→ Upsert schemas

→ Upsert tables

→ Upsert columns

→ Refresh projection table

14. Projection Refresh Strategy

When metadata changes occur:

```
DELETE FROM column_catalog_editor  
WHERE workspace_id = ?
```

```
INSERT INTO column_catalog_editor  
SELECT ...  
FROM schemas  
JOIN tables  
JOIN columns  
LEFT JOIN term_column_mapping  
LEFT JOIN business_terms
```

15. Spreadsheet Metadata Editor

Grid Columns

Schema

Table

Column

Example Value

Owner

Description

Business Term

Features

- Inline editing
- Keyboard navigation
- Filtering

- Sorting
- Bulk paste
- Optimistic updates

16. Bulk Metadata Updates

Endpoint:

PATCH /metadata/columns

This endpoint accepts BULK updates.

Request example:

```
[  
  {  
    column_id,  
    description,  
    example_value,  
    owner,  
    version  
  }  
]
```

17. Business Terms

Endpoints:

POST /business-terms

GET /business-terms

POST /business-terms/map

GET /business-terms returns all terms for the workspace
so the frontend can populate dropdown selections.

Example response:

```
[  
  {  
    id,  
    name,  
    definition  
  }  
]
```

Column-term relationships use:

term_column_mapping

Columns may belong to multiple terms.

18. Metadata Audit Log

All metadata edits must write to metadata_changes.

Triggered by:

PATCH /metadata/columns

POST /business-terms

POST /business-terms/map

19. Documentation Coverage

Endpoint:

GET /metadata/coverage

Returns workspace-level coverage.

Response:

```
{  
  total_columns,  
  documented_columns,  
  coverage_percent  
}
```

20. Demo Dataset (Seed Data)

Workspace seeded with demo dataset:

Acme Commerce Analytics

Schemas:

core

analytics

Tables and Example Columns (with types):

core.customers

- customer_id (uuid)
- email (text)
- first_name (text)
- last_name (text)
- signup_date (timestamp)

core.orders

- order_id (uuid)
- customer_id (uuid)
- order_total (numeric)
- order_date (timestamp)

core.order_items

- order_item_id (uuid)
- order_id (uuid)
- product_id (uuid)
- quantity (integer)
- item_price (numeric)

core.products

- product_id (uuid)
- product_name (text)
- category (text)
- price (numeric)

analytics.customer_lifetime_value

- customer_id (uuid)
- lifetime_value (numeric)
- last_purchase_date (timestamp)

analytics.daily_revenue

- revenue_date (date)
- total_revenue (numeric)
- order_count (integer)

analytics.product_sales

- product_id (uuid)
- units_sold (integer)
- revenue (numeric)

analytics.marketing_attribution

- campaign (text)
- channel (text)
- attributed_revenue (numeric)

Some columns should start with descriptions populated
and others left undocumented so coverage metrics are visible.

21. Performance Requirements

System must support 100k+ columns.

Required optimizations:

- Projection table reads
- Pagination (LIMIT 200)
- Indexed filters
- Batched updates
- Optimistic UI updates
- Row versioning

22. API Endpoints

GET /invite/{token}

POST /invite/{token}/join

POST /metadata/upload

GET /metadata/columns

PATCH /metadata/columns

POST /business-terms

GET /business-terms

POST /business-terms/map

GET /metadata/coverage

23. Non-Goals (MVP)

Do NOT implement:

- Authentication providers

- Database connectors
- Lineage engines
- Governance workflows
- RBAC
- AI assistants